

Záródolgozat

SZOFTVERFEJLESZTŐ ÉS TESZTELŐ

One Knight Army



Készítette: Bacskai Bence 13.D

Csapattársak: Juhász Márk Ferenc, Bangó Máté

Konzulens: Berdó Tamás

Aszód

2026

Nyilatkozat

Alulírott Bacskai Bence tanuló kijelentem, hogy a szakdolgozat teljes mértékben a saját munkám eredménye. A felhasznált eszközöket és szakirodalmakat (tanulmány, internetes forrás, személyes közlés stb.) idézőjel és megjelölt források nélkül nem építettem be, egyértelműen megjelöltem. Az elkészült záródolgozatban található eredményeket a képzőintézmény és a feladatot kiíró intézmény saját céljára térítés nélkül felhasználhatja, a titkosításra vonatkozó esetleges megkötések mellett.

Aszód, 2026. Bacskai Bence

Váci SZC PETŐFI SÁNDOR Műszaki Technikum, Gimnázium és Kollégium

KONZULTÁCIÓS LAP

Szoftverfejlesztő és tesztelő

A tanuló neve: Bacsikai Bence Osztálya: 13/D

A záródolgozat témája: Játékfejlesztés, a One Knight
Angry című játék megalkotása és fejlesztése
Unity motorban.

Sorszám:	A konzultáció időpontja:	A konzulens aláírása:
1.	2026. jan. 18	Bacsikai Bence
2.	2026. feb. 22	Bacsikai Bence
3.	2026. ápr. 03.	Bacsikai Bence
4.		
5.		

A záródolgozat beadható.
Aszód,

Bacsikai Bence
konzulens/

Tartalomjegyzék

Bevezetés	6
1. Felhasználói dokumentáció	7
1.1 Rendszerkövetelmények	7
1.1.1 Hardver követelmények	7
1.1.2. Szoftver követelmények	7
1.1.3. A program telepítése	8
1.2. A játék használatának részletes leírása	9
1.2.1. Főmenü	9
1.2.2 Beállítások	12
1.2.3 Karakter irányítása és mozgás	15
1.2.4 Harcrendszer és harcmodok	16
1.2.5 A játéktér és a felhasználó mutatói	18
1.2.6 Hős statisztikái és fejlődése	23
1.2.7 Skill Tree: Alakítsd a sorsodat	24
2. Fejlesztői dokumentáció	27
2.1 Témaválasztás	27
2.2. Alkalmazott fejlesztői eszközök	27
2.2.1. Programok, szoftverek, alkalmazások	27
2.3. Adatmodell leírása	30
2.3.1. Fő adatszerkezetek specifikációja	30
2.3.2. OOP architektúra és osztálykapcsolatok (UML előkészítés)	31
2.3.3. Adatkapcsolat leírása	31
2.4. Részletes feladatspecifikáció, algoritmusok	32
2.4.1. Player Movement	32
2.4.2. Player Combat	34
2.4.3. Player Health	35
2.4.4. ExpManager	36
2.4.5. Player_ChangeEquipment	37
2.4.6. Player_Bow	38
2.4.7. Arrow	39
2.4.8. Player_ChangeToGuard	40
2.4.9. Player_Heal	42

2.4.10.	Player_Rage	43
2.4.11.	StatsManager	45
2.4.12.	StatsUI	47
2.4.13.	Enemy_Movement	48
2.4.14.	Enemy_Health	50
2.4.15.	Enemy_Combat	51
2.4.16.	Enemy_Knockback	52
2.4.17.	SkillTreeToggler	53
2.4.18.	SkillSlot	54
2.4.19.	SkillTreeManager	55
2.4.20.	SkillManager	56
2.5.	Algoritmusok leírása (Pszudokód)	57
2.5.1	Szintlépési és Tapasztalati Pont Rendszer (ExpManager)	57
2.5.2	Fegyverváltás és Animációs Rétegek (Player_ChangeEquipment)	58
2.5.3	Ellenség Mesterséges Intelligencia (Enemy_Movement)	59
2.5.4.	Képességholdás Logika (SkillManager / SkillTreeManager)	60
2.6.	Tesztesetek bemutatása	61
2.6.1.	Normál tesztet: Szintlépés és Képességholdás	61
2.6.2.	Extrém tesztet: "Bolondbiztosság" – Képesség túlhúzása	61
2.6.3	Határérték tesztet: Pajzs és Életerő interakció	62
2.6.4	A tesztelés során kiderült hibák felsorolása	62
2.6.5.	Alkalmazott tesztelési módszertanok	63
2.7.	Jövőbeli fejlesztési lehetőségek	64
2.7.1.	Mesterséges Intelligencia és Ellenség-típusok	64
2.2.	Kibővített Képességfa (Skill Tree)	64
2.3.	Környezeti interakciók és Világépítés	64
Összegzés		65
Forrásmegjelölés		66
Ábrajegyzék		67

Bevezetés

A **szakdolgozatom témája** egy olyan videojáték fejlesztése, amely egyedi megközelítéssel törekszik a játékos figyelmének folyamatos fenntartására. A projekt címe **„One Knight Army”**, amely a stratégiai **„Tower Defense”** műfajt ötvözi az akcióelemekkel. A fejlesztés a game development szakirány keretében valósul meg, célunk pedig egy olyan dinamikus játékelmény létrehozása, amely a vizuális világ és a mechanikák révén maradandó élményt nyújt.

Játékmenet és Világ

A **One Knight Army** különlegessége, hogy a hagyományos Tower Defense játékokkal ellentétben a játékos nem csupán külső szemlélő, hanem egy lovag szemszögéből, aktív résztvevőként irányítja az eseményeket. A feladat a bázis védelme a hullámokban érkező ellenségtől, miközben a stratégiai védekezést és a karakter közvetlen irányítását egyensúlyban kell tartani.

Csapatmunka és Feladatmegosztás

A projektet három fős csapatban készítettük: Bangó Mátéval és Juhász Márk Ferencel. A közös munka baráti légkörben, rendkívül gördülékenyen és hatékonyan zajlott. A fejlesztési feladatokat az alábbiak szerint osztottuk meg:

- **Személyes hozzájárulásom:** Én feleltem a játszható karakter (a lovag) és az ellenségek fizikai viselkedéséért, valamint a komplex mozgásrendszer kidolgozásáért. A fejlődés és a talentum fa megalkotásáért is én feleltem.
- **Juhász Márk Ferenc:** Az ő munkáját dicséri a játék főmenüje, illetve a hozzá tartozó beállítási felületek kialakítása. A bolt rendszer megvalósítása valamint a pálya elkészítése is az ő feladata volt. A játék csata eseményének megírása.
- **Bangó Máté:** A játék hangjaiért valamint a játék 2 nyelvű szövegéért felelt. Ezek mellett csinált egy weboldalt ahol a játék letöltése van kifejtve

1. Felhasználói dokumentáció

1.1 Rendszerkövetelmények

1.1.1 Hardver követelmények

	Minimális	Ajánlott
Memória	4GB	8GB
Videókártya	Integrált (Intel HD 5000)	Nvidia GTX 960
Processzor	Intel i3 4000	Intel i3 6000
Kijelző felbontás	1280x720	1920x1080
Tárhely	1GB	5GB

1.1.2. Szoftver követelmények

Specifikáció	Minimális verzió
Direct X	11/12
Operációs rendszer	Windows 10 64bit/+
Visual C++ Redistributable	✓

1.1.3. A program telepítése

Letöltés: Látogasson el a projekt hivatalos weboldalára a <https://bacsek.itch.io/oneknightarmy> címen. Kattintson a „Download” gombra, majd mentse el a „Game.zip” nevű tömörített állományt a számítógépére.

One Knight Army

[More information](#) ▾

Download

Download

One-Knight_Army 47 MB

Comments

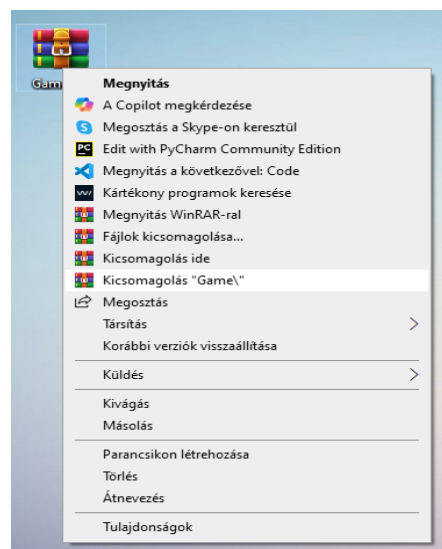
Write your comment...



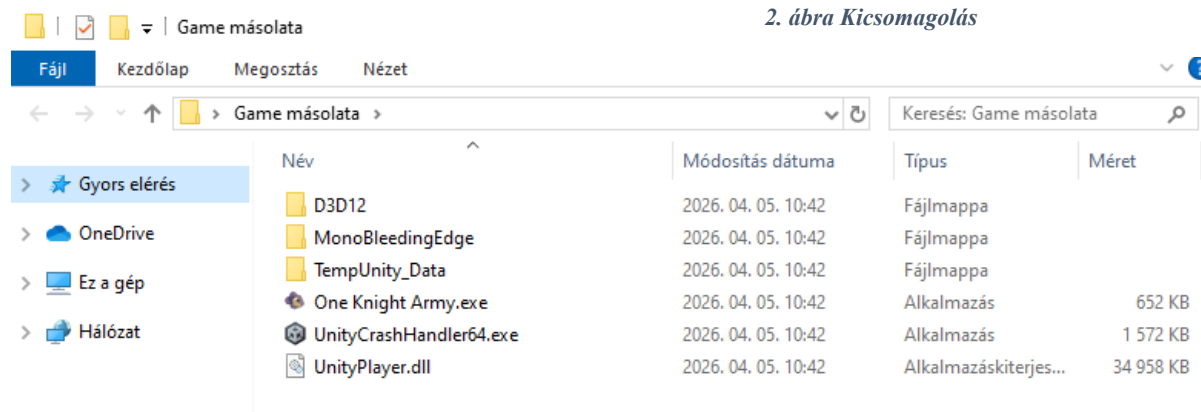
1. ábra Letöltés

Kicsomagolás: Keresse meg a letöltött fájlt, majd egy archiváló program segítségével (pl. WinRAR, 7-Zip vagy a Windows beépített varázslója) csomagolja ki a tartalmát egy tetszőleges mappába.

Indítás: Lépjen be a kicsomagolt mappába, és indítsa el a „One Knight Army.exe” végrehajtható fájlt. A játék ezt követően automatikusan elindul.



2. ábra Kicsomagolás



3. ábra Elindítás

1.2. A játék használatának részletes leírása

1.2.1. Főmenü

A Játék Címe: A képernyő felső részén megtalálható az indákkal átölelt logó amely a játék címét is magába foglalja.

A Hős: Bal oldalon látható a bátor, lila páncélos lovagot, aki készen áll a harcra.

Az Ellenség: Jobb oldalon egy hatalmas, zöld ogre figyeli a mozdulataidat, előrevetítve a rád váró kihívásokat.

Navigációs Gombok: Középen három jól elkülöníthető, interaktív gomb kapott helyet



4. ábra Főmenü

START GAME gomb:

Kiválasztás: A kurzor a gomb felülete fölé történő irányítását követően észreveheti, hogy a gomb színe megváltozott, jelezve, hogy készen áll az interakcióra.

Interakció: Helyezze a mutatóujját az **egér bal gombjára**, majd határozott mozdulattal nyomja le azt.

Eredmény: Ebben a pillanatban a rendszer feldolgozza a parancsot, a menürendszer háttérbe vonul, és kezdetét veszi a tényleges **játékmenet**.



5. ábra Start Game gomb

SETTINGS gomb:

Kiválasztás: Irányítsa a kurzort a SETTINGS feliratú zöld szalagra.

Interakció: Kattintson az **egér bal gombjával**.

Eredmény: Ekkor a főmenü háttere előtt egy új ablak vagy menüfelület nyílik meg.



6. ábra Settings gomb

EXIT gomb:

Kiválasztás: Irányítsa a kurzort az EXIT feliratú zöld szalagra.

Interakció: Kattintson rá az **egér bal gombjával**.

Eredmény: A kattintás hatására a program futása azonnal leáll, és az alkalmazásablak bezárul.



7. ábra Exit gomb

Egyéb vizuális látványelemek:

Belebegés: A játék betöltésekor a főmenü tartalma szépen lassan lebeg a képernyőre.

Élő háttér: A kurzor össze vissza mozgatása esetén a háttér és a karakterek élő hatást keltenek.



8. ábra Lebegő játékcím

1.2.2 Beállítások

Részletes vezérlés:

Miután a főmenüben a **SETTINGS** gombra kattintottál, ez a panel tárul eléd. Ezen a felületen összesen **5-féle interakciót** hajthatsz végre a játékelmény optimalizálása érdekében.



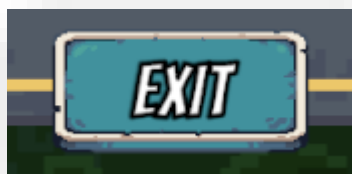
9. ábra Beállítások menü

EXIT gomb:

Kiválasztás: Irányítsa a kurzort az EXIT feliratú zöld szalagra.

Interakció: Kattintson rá az **egér bal gombjával**.

Eredmény: A kattintás hatására a program visszavisz a főmenüre.



10. ábra Exit gomb

GRAPHICS QUALITY:

A kép egy klasszikus **Graphics Quality** (Grafikus minőség) beállító panelt mutat be, amely a játékosnak lehetővé teszi a vizuális beállítások módosítását.

A panel két fő részből áll:

Aktuális állapot kijelző (felső sor)

Legördülő opciólista (alsó panel)



11. ábra Graphics Quality Selector

Normál működés: A rákattintva a felső „ULTRA” gombra legördül a lista kiválaszt egy elérhető opciót és azonnal frissül a játék grafikus beállítása. A játék automatikusan „Ultra” szintre van állítva. A többi kiválasztható lehetőségre kattintva a játék a kiválasztott minőségre vált ez csökkentheti és növelheti a játék által felhasznált erőforrásokat.

MASTER VOLUME:

Beállítás: Kattintson a kard pengéjén található kék jelzőre, majd húzza azt vízszintesen a kívánt irányba. Ez a csúszka a játékbéli összes hanghatást változtatja.

Halkítás / Némítás: A csúszka balra történő mozgatásával a hangerő csökken. A bal szélső pozíció teljes némítást eredményez.

Hangosítás: A csúszka jobbra történő mozgatásával a hangerő növekszik.



MUSIC VOLUME:

Beállítás: Kattintson a kard pengéjén található kék jelzőre, majd húzza azt vízszintesen a kívánt irányba. Ez a csúszka a játékbeli összes aláfető zenének a hangerejét változtatja.

Halkítás / Némítás: A csúszka balra történő mozgásával a hangerő csökken. A bal szélső pozíció teljes némítást eredményez.

Hangosítás: A csúszka jobbra történő mozgásával a hangerő növekszik.



12. ábra Music Volume Slider

SFX VOLUME:

Beállítás: Kattintson a kard pengéjén található kék jelzőre, majd húzza azt vízszintesen a kívánt irányba. Ez a csúszka a játékbeli összes hangeffektust mind például a kard suhintását változtatja.

Halkítás / Némítás: A csúszka balra történő mozgásával a hangerő csökken. A bal szélső pozíció teljes némítást eredményez.

Hangosítás: A csúszka jobbra történő mozgásával a hangerő növekszik.



13. ábra SFX Volume Slider

1.2.3 Karakter irányítása és mozgás

A hős mozgatása többféle eszközzel is lehetséges, így kiválasztja a számára legkényelmesebb irányítási módot a kaland során.

Alapvető mozgás billentyűzettel:

A karaktert a **W**, **A**, **S**, **D** billentyűkkel vagy az **Iránybillentyűkkel (nyilak)** terelheted a kívánt irányba:

Felfelé haladás: Nyomd meg a **W** vagy a **Felfelé nyíl (↑)** billentyűt.

Lefelé haladás: Használd az **S** vagy a **Lefelé nyíl (↓)** billentyűt.

Balra haladás: A hős az **A** vagy a **Balra nyíl (←)** lenyomására indul el balra.

Jobbra haladás: A haladáshoz nyomd meg a **D** vagy a **Jobbra nyíl (→)** billentyűt.

Navigációs tippek:

Átlós mozgás: Billentyűzetten két egymásra merőleges irány (például **W** és **D**) egyidejű lenyomásával a hősöd 45 fokos szögben is képes haladni.



14. ábra A Hős

1.2.4 Harcrendszer és harcmódok

A lovagja nemcsak a védekezésben jártas, hanem sokoldalú harcos is, aki képes alkalmazkodni a távolsági és a közelharci helyzetekhez egyaránt.

Támadás és akciógombok:

A támadások kivitelezése mindkét harcmódban egyszerű és gyors:

Támadás / Lövés: Használja a **Space (Szóköz)** billentyűt az aktív fegyvere használatához.

Közelharc: Harcos módban a **Space** lenyomásával a lovag a kardjával csap le a közvetlen közelében lévő ellenségekre.

Távolsági harc: Íjász módban a **Space** lenyomásával a lovag nyílvezzőt lő ki a nézési irányába.

Váltás a harcmódok között:

A hatékony harc kulcsa a gyors fegyverváltás, amellyel bármilyen szituációt uralni tud:

Módváltás: Nyomja meg a **Q** billentyűt a **Harcos** (kard és pajzs) és az **Íjász** (távolsági fegyver) módok közötti váltáshoz.

Stratégia: Váltson íjászra a távolról közelítő ellenségek gyengítéséhez, majd gyorsan váltson vissza harcosra a **Q** gombbal, ha az ellenfél eléri a közelharci távolságot.



15. ábra Kard Csapás



16. ábra Íjász Mód



17. ábra Íj Lövés

Védekezés és harci mechanika:

A túlélés érdekében nemcsak a támadásra, hanem a pontos időzítésre és a pajzs hatékony használatára is szüksége lesz.

Védekezés és háritás:

A hős képes teljesen semlegesíteni az ellenséges csapásokat, ha megfelelően használja a pajzsát:



18. ábra Guard/Block

Védekezés: Tartsa lenyomva a **Bal Shift** billentyűt a pajzs felemeléséhez.

Támadás kivédése: Ha a pajzs aktív, a szemből érkező támadásokat a lovag kivédi, így nem veszít életerőt.

Visszatöltési idő (Cooldown): Sikeres blokkolás után egy időcsík jelenik meg a karakter felett vagy a felületen. Ez jelzi a védekezés újratöltődését; amíg a csík be nem tölt teljesen, nem lehet újra használni a pajzsot.



19. ábra Blokk Rúd

Összetett harci mozdulatok:

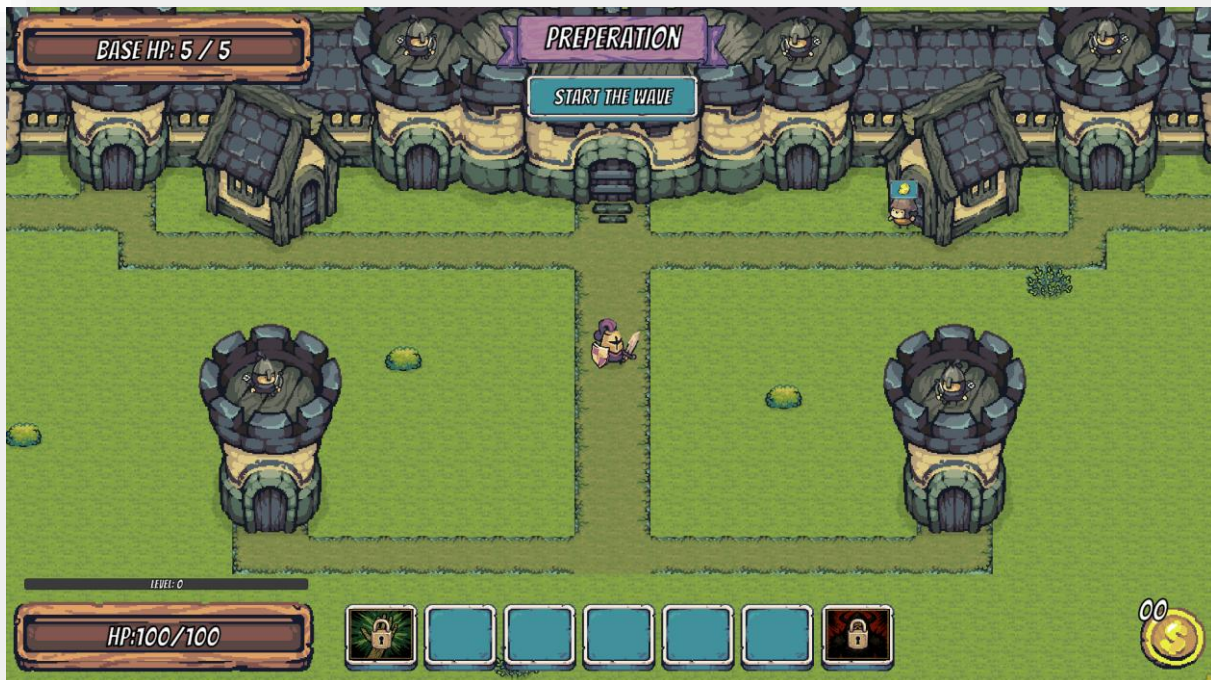
A billentyűzet kombinálásával dinamikus harcmodort alakítható ki:

Váltás és védelem: A Q billentyű használatával a harcos módba való visszatéréshez, ha védekezni kényszerül, mivel az íjász módban a pajzs nem használható.

Támadás (Space): A védekezések közötti szünetekben a Space billentyűvel (kardcsapás vagy lövés) viheti be a döntő találatokat.

1.2.5 A játéktér és a felhasználó mutatói

Ebben a fejezetben áttekintjük a csatater felépítését és azokat a kritikus jelzőket, amelyek a túlélést és a stratégiai döntéseket segítik a játék során. Az alábbi rendszerek ismerete alapvető fontosságú a sikeres védekezéshez:



20. ábra Pálya

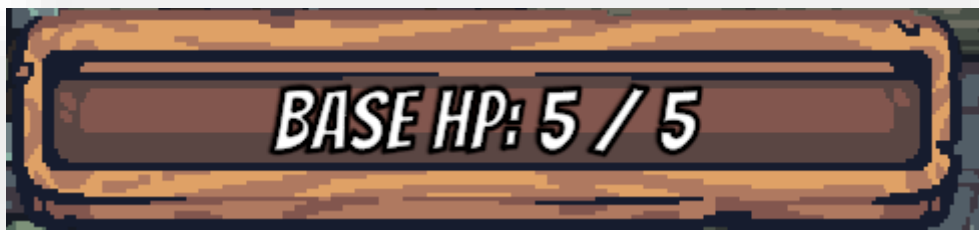
1.2.5.1. A felület (HUD) elemei

Hős életereje: A bal alsó sarokban látható **HP: (100 / 100)** sáv mutatja a karakter aktuális és maximális élet erejét.



21. ábra Életerő jelző

Bázis állapota: A bal felső sarokban található **Base HP (5 / 5)** jelzi az erődtítmény épségét; ha az ellenség áttöri a védelmet, ez az érték csökken.



22. ábra Bázis Életerő Jelző

Tapasztalati szint: Az életerő sáv feletti Level mutató jelzi a hős fejlődését és aktuális szintjét.



23. ábra Tapasztalat Szint Jelző

Képességtár (Hotbar): Az alsó középső foglalatokban láthatók az aktív és még lezárt képességek, melyek a harci hatékonyságot növelik.



24. ábra Ezköztár

Aranytartalék: A jobb alsó sarokban található az összegyűjtött érmék száma, amelyeket kereskedésre lehet fordítani.



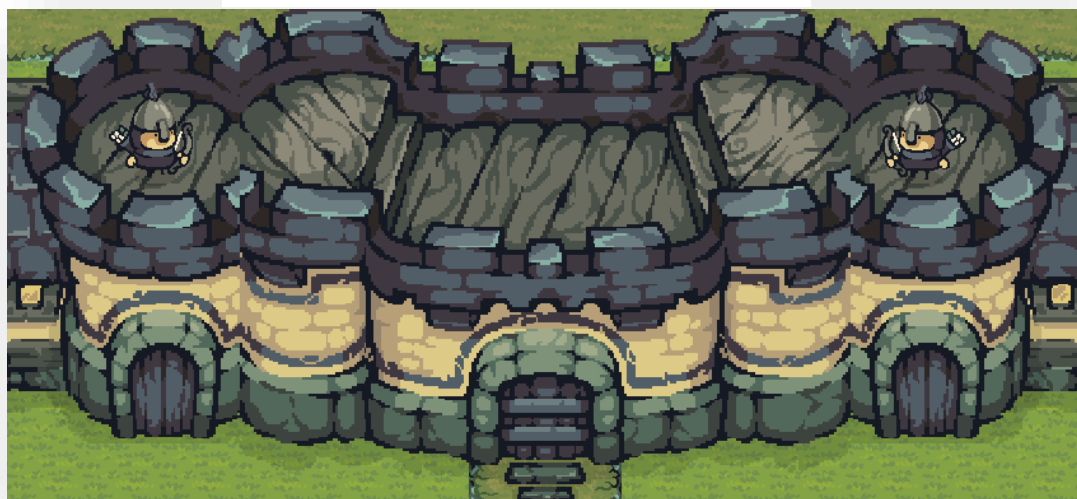
25. ábra Arany Jelző

1.2.5.2. Játéktér és környezet:

Védelmi épületek: A területen stratégiailag elhelyezett **őrtornyok** és várfalak találhatók, amelyeken íjászok segítik a védekezést.

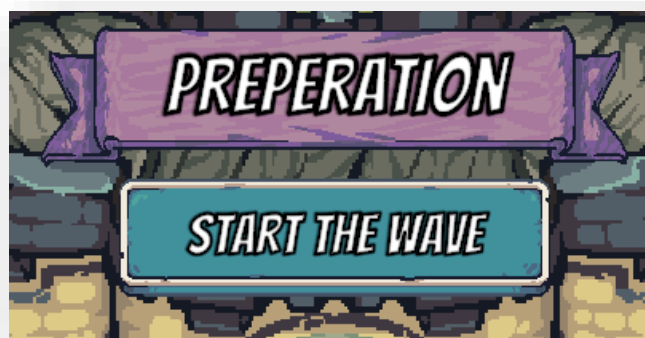


26. ábra Tornyok



27. ábra Várfal

Fázisjelző: A képernyő felső részén a **Preparation** felirat jelzi a felkészülési szakaszt, ahol a **Start the Wave** gombbal indíthatod el a következő támadást.



28. ábra Csata Indítása Gomb

1.2.5.3. Kereskedelem

A bolt elérése és használata

Interakció: Ha a karakter egy boltos mellé lép, az **E** billentyű megnyomásával megnyitható a kereskedelmi felület.

Kínálat: A boltokban különféle főzeteket (potion) lehet vásárolni, amelyek segítenek a túlélésben és a harci hatékonyság növelésében.

Vizuális visszajelzés: A látogatható épületeket egy ikon jelzi, így könnyen felismerhető a kereskedő játéktéren.



29. ábra Boltos

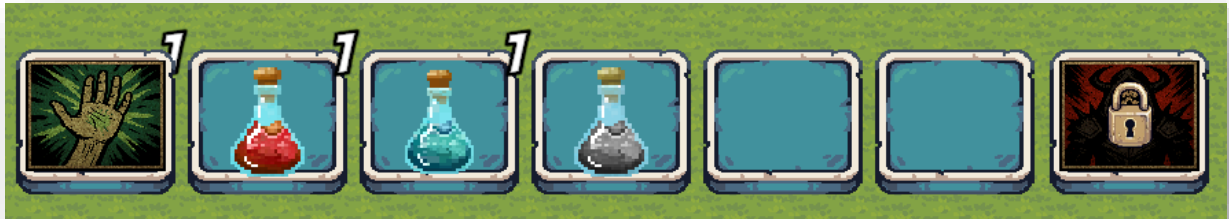
Vásárolható főzetek listája:

- **Heal Potion (Gyógyító főzet)(Piros):** Visszatölti a hős elveszített életerejét, hogy tovább bírja a harcot.
 - **Ára: 30 arany.**
- **Swiftess Potion (Gyorsaság főzete)(Kék):** Ideiglenesen megnöveli a karakter mozgási sebességét, segítve a gyorsabb pozicionálást vagy a menekülést.
 - **Ára: 60 arany.**
- **Extra HP (Extra életerő)(Szürke):** Megnöveli a hős maximális életerő keretét, így szívósabb harcosná válik az ostrom alatt.
 - **Ára: 50 arany.**

Hasznos tudnivalók:

Vásárlás folyamata: Válassza ki a kívánt főzetet a bolt menüjében, és győződjön meg róla, hogy rendelkezik a jobb alsó sarokban látható elegendő aranyösszeggel. Kattintást követően lekerül egy példány az imént említett képességtárba.

Főzetek használata: A már képességtárban lévő Főzetekre kattintva aktiválhatja őket.



30. ábra Ezköztár

Kereskedői üdvözlét: A boltos barátságos fogadtatása ("It's the greatest pleasure to see you return alive") jelzi, hogy biztonságban vagy, amíg a menü nyitva van.



31. ábra Bolt Megnyitva

1.2.6 Hős statisztikái és fejlődése

A csaták szünetében érdemes figyelemmel kísérni a hős fizikai állapotát és képességeit, hogy mindig készen álljon a növekvő kihívásokra.

A Stats (Statisztikák) menü elérése

Megnyitás: A T billentyű lenyomásával bármikor előhívható a hős adatait tartalmazó áttekintő ablak.

Alapvető tulajdonságok: Ebben a menüben nyomon követhető a karakter három fő pillérje:

Strength (Erő): Befolyásolja a sebzés mértékét.

Agility (Ügyesség): Meghatározza a támadási sebességet.

Vitality (Életerő): Növeli a túlélési esélyeket és a maximális életerőt.

Tapasztalati szint: A menü alján látható Level sáv jelzi, mennyire áll közel a hős következő szintlépéshez.

Skill Tree elérése: Ha a statisztikai ablakban közvetlenül a hős ikonjára kattintva, a játék átirányít a Skill Tree (Képességfa) felületre.



32. ábra Statisztika

1.2.7 Skill Tree: Alakítsd a sorsodat

A szintlépések során szerzett tapasztalat itt váltható valódi erőre. A képességfa lehetővé teszi, hogy a hős felbírja venni a versenyt az erősödő akadályokkal és ellenfelekkel.

Fejleszthető képességek és hatásaik:

- **Sebzésnövelés (Kard ikon):** Növeli a közelharc támadások erejének nagyságát. [10 fokozat] [1 fokozat= plusz 5 sebzés]
- **Életerőnövelés (Szív ikon):** Tartósan megnöveli a hős maximális életerejét. [10 fokozat] [1 fokozat= plusz 10 maximális életerő]
- **Gyorsaságnövelés (Szárnyas csizma ikon):** Állandó bónuszt ad a mozgási sebességhez. [5 fokozat]
- **Íjászat fejlesztése (Nyílzápor ikon - lakat alatt):** Növeli a távolsági támadások sebzését és sebességét. [5 fokozat]

Az alsó sorban a speciális technikák és kiegészítő készségek kaptak helyet:

- **Íjászat feloldása (Íj ikon):** Ezzel a ponttal nyílik meg a lehetőség, hogy harc közben újra váltson a hős. [1 pont]
- **Blokkolás feloldása (Pajzs ikon):** Lehetővé teszi a pajzs aktív használatát a támadások kivédéséhez. [1 pont]
- **Gyógyító képesség fejlesztése (Kéz ikon):** Feloldja majd fejleszti a gyógyítás képességet. [5 fokozat]
- **Düh képesség feloldása (Démoni alak ikon - lakat alatt):** Egy speciális állapotot tesz elérhetővé, amellyel rövid ideig sebezhetetlenné válik a hős. [5 fokozat kell a feloldásához]



33. ábra Talentumfa

Pontok elosztása és kezelés:

- **Points (Pontok):** A képernyő alján látható, hány elkölthető képességpont van. Pontokat minden szintlépés után kap a hős.
- **Fejlesztés:** Kattintson a kívánt ikonra a pont elköltéséhez.
- **Kilépés:** Az Exit gombbal térhet vissza a statisztikákhoz.

Gyógyító képesség (Kéz ikon): Feloldás után a képességtárban használhatóvá válik. Használat esetén a karakter visszatölt egy kevés életerőt, majd egy visszatöltési időre kerül amit egy szürke töltés mutat.



34. ábra Gyógyító Képesség Gomb

Düh képesség (Démoni alak ikon): Feloldás után a képességtárban használhatóvá válik. Használat esetén rövid ideig sebezhetetlenné válik a hős amit egy piros csúszka jelez,



36. ábra Düh Képesség Gomb



35. ábra Düh Képesség Jelző

ezen idő letelte után elindul a visszatöltési idő.

2. Fejlesztői dokumentáció

2.1 Témaválasztás

A szakdolgozatom témája a **One Knight Army**, egy egyedi Tower Defense stratégiai játék, amelyben a játékos egy lovag szemszögéből irányítja az eseményeket. Célunk egy olyan élmény létrehozása volt, amely a dinamikus játékmenet és az interaktív narratíva segítségével végig fenntartja a játékos figyelmét.

2.2. Alkalmazott fejlesztői eszközök

2.2.1. Programok, szoftverek, alkalmazások

-Unity

A projekt elsődleges fejlesztői környezete és játékmotorja.

-Visual Studio 2022

A fejlesztéshez használt integrált fejlesztői környezet (IDE), amelyben a játék teljes logikája, a fizikai számítások és a karakterek mozgásáért felelős C# szkriptek készültek.

-TilemapEditor

Ezzel a Unity funkcióval készült el a pálya egésze.

-Cinemachine

A Unity beépített funkciója amelyel könnyen létrehoztam egy kamerát amely követi a játszott karaktert.

-Animator

Alapvető Unity funkció amely segítségével életre tudjuk hozni a karaktereket és animálni őket.

-Tiny swords

A játék asset pack-je ez tartalmazza a játékon belül látott környezetet és karaktereket.

-Git / GitHub / GitLab

Verziókezelő rendszer, amely lehetővé tette a projekt mentését, a kódmódosítások visszakövetését és a biztonságos fejlesztést.

-Bfxr / Audacity

A játék hangeffektjeinek (például a kardcsapás vagy a főzet vásárlásakor hallható hangok) előállítására és vágására szolgáló ingyenes eszközök.

-Google Fontok / DaFont

A kezelőfelületen (például a „Preparation” vagy „Stats” feliratoknál) használt egyedi betűtípusok forrása.

-TextMeshPro

A Unity beépített szövegkezelője, amely lehetővé tette a „Preparation”, „Stats” és „Skill Tree” feliratok túéles és stílusos megjelenítését.

-Physics 2D Engine

A Unity beépített fizikai motorja, amely a lövedékek pályájáért és az ellenségekkel való ütközésérzékelésért felel.

-IL2CPP / Mono

A Unity fordítóprogramjai, amelyek a Visual Studio-ban írt C# kódot a gép számára értelmezhető és futtatható állománnyá alakítják.

2.3. Adatmodell leírása

A projekt nem klasszikus relációs adatbázist (mint az SQL) használ, hanem a **Unity Engine** objektumorientált adatkezelési rendszereire épül. Az adatok tárolása, módosítása és mentése szétválasztott osztályok és adatkonténerek segítségével valósul meg, biztosítva a moduláris felépítést.

2.3.1. Fő adatszerkezetek specifikációja

A játék logikája négy központi adatszerkezet köré csoportosul, amelyek meghatározzák a játékmenet menetét és a hős fejlődését.

PlayerStats (Dinamikus karakteradatok)

Ez az adatszerkezet felel a hős pillanatnyi állapotának követéséért. Nem csupán statikus számokat tárol, hanem eseményeket is generál (pl. ha a HP változik, a UI frissül).

Változók: **currentHP** / **maxHP** (float): A bal alsó sarokban látható életerő-sáv alapja.

currentLevel (int): A fejlődési szint, amely meghatározza a megszerzhető képességpontokat.

experiencePoints (float): A szintlépéshez szükséges haladási érték.

goldBalance (int): A jobb alsó sarokban látható fizetőeszköz mennyisége.

SkillTree hierarchia

A képességfa (T billentyű -> hős ikon) egy összefüggő logikai lánc. Minden képesség (sebzés, gyorsaság, íjászat stb.) egy egyedi azonosítóval rendelkezik.

2.3.2. OOP architektúra és osztálykapcsolatok (UML előkészítés)

A program felépítése követi a **Model-View-Controller (MVC)** elv egyszerűsített változatát, ahol az adatok (Stats) és a megjelenítés (UI) szigorúan elválnak egymástól.

- **CombatManager:** Ez az osztály felel a **Space** billentyű (támadás) és a **Bal Shift** (védekezés) logikai összekötéséért. Figyeli a védekezés utáni **időcsíkot (cooldown)**, és letiltja a pajzsot, amíg az le nem jár.
- **ShopSystem:** Kezeli az interakciót az NPC-kel. Ellenőrzi a távolságot a játékos és a boltos között, majd az **E** lenyomásakor összeveti a goldBalance-t a tárgy árával.
- **UIManager:** Központi figyelő egység. Ha a **Base HP** értéke változik a bal felső sarokban, ez az osztály frissíti a grafikus felületet.

2.3.3. Adatkapcsolat leírása

- **Egy-a-többhöz kapcsolat:** A WaveSystem több EnemyInstance-t hoz létre, melyek mindegyike a BaseEntity adatait használja.
- **Függőségi kapcsolat:** A SkillTree közvetlen eléréssel rendelkezik a StatManager-hez. Amikor a játékos elkölt egy pontot (pl. Strength növelés), a SkillTree meghívja a StatManager.UpdateStats() függvényt.
- **UI-Adat szinkron:** A HUD osztály folyamatosan "fel van iratkozva" a játékos adatváltozásaira, így minden aranygyűjtés vagy sebződés azonnal megjelenik a kijelzőn.

2.4. Részletes feladat-specifikáció, algoritmusok

2.4.1. Player Movement

Update()

Leírás: Minden képkockánál (frame) meghívódik. Figyeli a játékos bevitelét: ha a játékos megnyomja a "Slash" gombot, aktiválja a Player_Combat szkripten keresztül a támadást.

FixedUpdate()

Leírás: Meghatározott időközönként (fizikai frissítéskor) fut le. Kezeli a karakter mozgását, a vízszintes és függőleges irányítást, valamint a "Flip" (megfordulás) funkciót. Ha a játékos éppen lő, támad vagy hátralökődött (Knockback), a mozgás korlátozva van. Itt frissülnek az animátor paraméterei és a támadási pont (attackPoint) pozíciója is a mozgás irányának megfelelően.

Flip()

Leírás: Megfordítja a karaktert a vízszintes tengelyen. Megváltoztatja a facingDirection értékét (1 vagy -1), és invertálja a karakter localScale.x értékét, hogy a sprite vizuálisan is a megfelelő irányba nézzon.

Knockback()

Leírás: Akkor hívódik meg, ha a karaktert hátralökik (például egy ellenséggel való ütközéskor). Kiszámítja az ellenségtől távolodó irányt, alkalmazza a fizikai erőt a Rigidbody2D-re, és elindítja a bénulási időt (stun) kezelő folyamatot.

Paraméterek:

- Transform enemy: Az ellenség pozíciója, aki a lökést okozza.
- float force: A lökés ereje.
- float stunTime: Az időtartam, amíg a játékos nem tud mozogni.

ResetMovement()

Leírás: Alaphelyzetbe állítja a játékos mozgási állapotait. Leállítja a hátralökődést, a lövést, nullázza a sebességet, és visszaállítja az animációs paramétereket (mozgás és támadás leállítása).

KnockbackCounter (Coroutine)

Leírás: Egy aszinkron várakozás, amely a megadott bénulási idő (stunTime) után nullázza a karakter sebességét és újra engedélyezi a mozgást (kikapcsolja az isKnockedBack állapotot).

Paraméterek: float stunTime: A várakozási idő másodpercben.

```
void FixedUpdate()
{
    if (isShooting)
    {
        rb.linearVelocity = Vector2.zero;
    }
    else if (!isKnockedBack)
    {
        float horizontal = Input.GetAxisRaw("Horizontal");
        float vertical = Input.GetAxisRaw("Vertical");

        bool currentlyAttacking = anim.GetBool("IsAttacking");

        if (!currentlyAttacking)
        {
            if (horizontal > 0 && transform.localScale.x < 0 || horizontal < 0 && transform.localScale.x > 0)
            {
                Flip();
            }

            Vector2 movement = new Vector2(horizontal, vertical).normalized;
            rb.linearVelocity = movement * StatsManager.Instance.speed;

            if (movement != Vector2.zero)
            {
                Vector2 newPos = movement * attackOffset;
                if (facingDirection == -1)
                {
                    newPos.x *= -1;
                }
                player_Combat.attackPoint.localPosition = newPos;
            }
        }
        else
        {
            rb.linearVelocity = Vector2.zero;
        }

        anim.SetFloat("horizontal", Mathf.Abs(horizontal));
        anim.SetFloat("vertical", Mathf.Abs(vertical));
    }
}
```

37. ábra Player Movement Script

2.4.2. Player Combat

Attack()

Leírás: Ez a metódus indítja el a támadási folyamatot. Amennyiben a támadások közötti várakozási idő (cooldown) lejárt, a függvény aktiválja a támadási animációt, rögzíti a karakter pozícióját a fizikai motorban (Rigidbody) a mozgás megakadályozása érdekében, és újraindítja az időzítőt.

DealDamage()

Leírás: Ez a metódus végzi el a tényleges sebzéskiosztást egy kör alakú tartományban (OverlapCircleAll) a támadási pont körül. A tartományon belül lévő összes ellenséget érzékeli, csökkenti az életerejüket a játékos aktuális sebzésének megfelelően, és ha az ellenség rendelkezik ilyen komponenssel, hátrálókést (knockback) alkalmaz rajtuk.

FinishAttacking()

Leírás: A támadási folyamat lezárásáért felelős metódus. Kikapcsolja a támadási animációt, és feloldja a karakter pozíciójának rögzítését a fizikai motorban, így a hős újra képes lesz a szabad mozgásra.

```
public void Attack()
{
    if (timer <= 0)
    {
        anim.SetBool("IsAttacking", true);
        rb.constraints = rb.constraints | RigidbodyConstraints2D.FreezePositionX | RigidbodyConstraints2D.FreezePositionY;

        timer = cooldown;
    }
}

public void DealDamage()
{
    Collider2D[] hitEnemies = Physics2D.OverlapCircleAll(attackPoint.position, StatsManager.Instance.weaponRange, enemyLayer);

    foreach (Collider2D enemy in hitEnemies)
    {
        if (enemy.TryGetComponent(out Enemy_Health health))
        {
            health.ChangeHealth((int)-StatsManager.Instance.damage);

            if (enemy.TryGetComponent(out Enemy_Knockback kb))
            {
                kb.Knockback(transform, StatsManager.Instance.knockbackForce, StatsManager.Instance.knockbackTime, StatsManager.Instance.stunTime);
            }
        }
    }
}

public void FinishAttacking()
{
    anim.SetBool("IsAttacking", false);
    rb.constraints = rb.constraints & ~RigidbodyConstraints2D.FreezePositionX & ~RigidbodyConstraints2D.FreezePositionY;
}
```

38. ábra Player Attack Script

2.4.3. Player Health

Awake()

Leírás: Az objektum példányosításakor hívódik meg. Inicializálja a belső referenciákat (mozgás szkript, sprite renderer, ütköző, rigidbody), hogy a későbbiekben hatékonyan tudja kezelni a játékos állapotait.

Start() / OnEnable()

Leírás: A szkript indulásakor vagy az objektum bekapcsolásakor fut le. Frissíti a felhasználói felületet (UI), hogy az aktuális életerő értékek jelenjenek meg.

ChangeHealth()

Leírás: Megváltoztatja a játékos életerejét. Kezeli a speciális állapotokat: figyelmen kívül hagyja a sebzést "Rage" módban, vagy blokkolja azt, ha a játékos éppen védekezik. Sebzés esetén képernyővillanást és rázkódást (Cinemachine Impulse) vált ki amelyet Juhász Márk Ferenc írt. Ha az életerő 0 alá esik, részecskeeffektet indít és meghívja a RespawnManager-t amelyet Juhász Márk Ferenc írt.

Paraméterek: int amount: Az életerő változás mértéke (pozitív gyógyulásnál, negatív sebzésnél).

Die()

Leírás: Elindítja a játékos halálát és újraszületését kezelő folyamatot (RespawnRoutine).

UpdateUI()

Leírás: Frissíti a képernyőn látható életerő szöveget a StatsManager-ben tárolt aktuális és maximális életerő értékek alapján.

RespawnRoutine (Coroutine)

Leírás: A játékos halála utáni várakozást és újraéledést vezérli. Kikapcsolja a mozgást és a megjelenítést, elteleportálja a karaktert, majd a várakozási idő lejárta után feltölti az életerőt és újra láthatóvá teszi a játékost.

2.4.4. ExpManager

OnEnable() / OnDisable()

Leírás: Kezeli az eseményfeliratkozásokat. Amikor a szkript aktívvá válik, feliratkozik az Enemy_Health.OnMonsterDefeated eseményre, így a szörnyek legyőzésekor automatikusan tapasztalati pontot kap a játékos. Kikapcsoláskor leiratkozik az eseményről a memóriaszivárgások elkerülése érdekében.

GainExperience()

Leírás: Növeli a játékos aktuális tapasztalati pontjait a megadott mennyiséggel. Ellenőrzi, hogy a pontok elérték-e a szintlépéshez szükséges határt; ha igen, meghívja a LevelUp metódust, majd frissíti a kezelőfelületet.

Paraméterek: int amount: A megszerzett tapasztalati pont mennyisége.

LevelUp()

Leírás: Kezeli a szintlépés folyamatát. Növeli a szintet, levonja a felhasznált pontokat, és az expGrowth szorzó alapján megemeli a következő szinthez szükséges ponthatárt. Végül elsüti az OnLevelUp eseményt, értesítve a játék más rendszereit a változásról.

UpdateUI()Leírás: Szinkronizálja a vizuális elemeket a belső adatokkal. Beállítja az expSlider maximum értékét és aktuális állását, valamint frissíti a szintet jelző szöveges mezőt.

```
private void Start()
{
    UpdateUI();
}

private void OnEnable()
{
    Enemy_Health.OnMonsterDefeated += GainExperience;
}

private void OnDisable()
{
    Enemy_Health.OnMonsterDefeated -= GainExperience;
}

public void GainExperience(int amount)
{
    currentExp += amount;
    if (currentExp >= expToLevel)
    {
        LevelUp();
    }
    UpdateUI();
}

private void LevelUp()
{
    level++;
    currentExp -= expToLevel;
    expToLevel = Mathf.RoundToInt(expToLevel * expGrowth);
    OnLevelUp?.Invoke(1);
}

public void UpdateUI()
{
    expSlider.maxValue = expToLevel;
    expSlider.value = currentExp;
    currentLvl.text = "Level: " + level;
}
```

39. ábra ExpManager Script

2.4.5. Player_ChangeEquipment

Awake()

Leírás: Az objektum inicializálásakor fut le. Ellenőrzi, hogy hozzá van-e rendelve az Animator komponens a változóhoz; ha nincs, automatikusan megkeresi és eltárolja azt a játékos objektumáról.

Start()

Leírás: A játék indításakor beállítja az alapértelmezett harci módot.

Update()

Leírás: Folyamatosan figyeli a játékos bemenetét. Ha a fegyverváltás engedélyezett (canChangeWeapon igaz), és a játékos megnyomja a "ChangeEquipment" gombot, meghívja a fegyverváltásért felelős metódust.

SwitchWeaponMode()

Leírás: Megcseréli a két harci módot: kikapcsolja az aktuális fegyverszkriptet és bekapcsolja a másikat. Ezzel párhuzamosan módosítja az Animator rétegsúlyait (Weight), hogy a karakter a megfelelő fegyverhez tartozó animációkat használja. Íjra váltáskor elindítja a lövési töltési időt (cooldown) és lejátssza a váltáshoz tartozó hangeffektust.

```
private void SwitchWeaponMode()
{
    bool switchingToBow = !bow.enabled;
    combat.enabled = !combat.enabled;
    bow.enabled = !bow.enabled;
    if (switchingToBow)
    {
        playerAnimator.SetLayerWeight(0, 0f);
        playerAnimator.SetLayerWeight(1, 1f);
    }
    else
    {
        playerAnimator.SetLayerWeight(0, 1f);
        playerAnimator.SetLayerWeight(1, 0f);
    }

    if (bow.enabled)
    {
        StatsManager.Instance.shootTimer = StatsManager.Instance.shootCooldown;
    }

    if (switchSound != null)
    {
        switchSound.Play();
    }
}
```

40. ábra Player_ChangeEquipment Script

2.4.6. Player_Bow

OnEnable() / OnDisable()

Leírás: Kezeli a távolsági harci mód aktiválását és deaktiválását. Bekapcsoláskor átállítja az animátor rétegeit az íjász animációkhoz, letiltja a védekezést (guard), és elindítja a lövési töltési időt. Kikapcsoláskor visszaállítja a karaktert az alapértelmezett (közelharc) animációs rétegre és újra engedélyezi a védekezést.

Update()

Leírás: Minden képkockánál frissíti a lövési időzítőt (shootTimer) és meghívja a célzásért felelős metódust. Figyeli a lövés gombot: ha a fegyver készen áll és lejárt a minimális várakozási idő az elővétel óta, elindítja a lövési animációt és rögzíti a lövési állapotot.

HandleAiming()

Leírás: Feldolgozza a játékos iránybeviteli adatait a célzáshoz. Ha van aktív irányítás, frissíti az aimDirection értékét, egyébként az utolsó ismert irányt tartja meg. Az aktuális célzási irányt (X és Y tengelyen) átadja az animátornak a megfelelő vizuális megjelenítéshez.

SnapToEightDirections()

Leírás: Egy segédfüggvény, amely a szabad célzási irányt a nyolc fő irány egyikéhez (fel, le, balra, jobbra és az átlók) igazítja 45 fokos ugrásokkal. Ez biztosítja a precízebb, arcade-stílusú lövést.

Paraméterek: Vector2 dir: Az eredeti, nyers irányvektor.

Visszatérési érték: Vector2 (a legközelebbi fő irányba kerekített vektor).

Shoot()

Leírás: Maga a lövés végrehajtása (általában animációs esemény hívja meg). Példányosítja a nyílveesszőt a launchPoint pozíciójában, beállítja annak irányát, sebességét és sebzését a statisztikák alapján. A lövés után újraindítja a töltési időt és felszabadítja a karakter mozgását.

2.4.7. Arrow

Start()

Leírás: A nyílvesző létrejöttkor (példányosításakor) hívódik meg. Beállítja a lövedék sebességét a megadott irányba, elforgatja a sprite-ot a repülési iránynak megfelelően, és elindítja az automatikus megsemmisítést az élettartam (lifespan) lejárta után.

RotateArrow()

Leírás: Kiszámítja a nyílvesző haladási iránya alapján a szükséges rotációt. A `Mathf.Atan2` segítségével meghatározza a szöget, majd a karakter/objektum Z-tengely körüli elforgatásával biztosítja, hogy a nyíl hegye mindig előre nézzen.

OnCollisionEnter2D()

Leírás: Kezeli az ütközéseket. Ha a nyíl ellenséget ér (LayerMask alapján), sebzést okoz az `Enemy_Health` komponensen keresztül, és hátralökést (Knockback) vált ki. Ha a nyíl akadályba (például falba) ütközik, meghívja az `AttachToTarget` metódust a becsapódás vizuális kezeléséhez.

Paraméterek: `Collision2D collision`: Az ütközés fizikai adatai és az eltalált objektum referenciája.

AttachToTarget()

Leírás: Kezeli a nyílvesző "beágyazódását" a célpontba. Lecseréli a grafikát a becsapódott állapotra (`buriedSprite`), leállítja a mozgást, a fizikai testet `Kinematic` típusúra váltja, és a nyilat az eltalált objektum gyerekobjektumává teszi, hogy azzal együtt mozogjon tovább.

Paraméterek: `Transform target`: Az objektum (akadály), amelybe a nyíl befűródött.

2.4.8. Player_ChangeToGuard

Start()

Leírás: Az objektum indulásakor hívódik meg. Beállítja a pajzs visszatöltődését jelző csúszka (Slider) maximum értékét és aktuális állását a megadott töltési idő alapján.

Update()

Leírás: Folyamatosan figyeli a "Guard" gomb állapotát. Ha a védekezés engedélyezett, a gomb lenyomásakor elindítja a védekezést (StartGuarding), a gomb felengedésekor pedig leállítja azt (StopGuarding).

StartGuarding()

Leírás: Aktiválja a védekező állapotot. Harmadára csökkenti a játékos mozgási sebességét, kikapcsolja a támadási lehetőséget, és az animátort a pajzshasználathoz szükséges rétegre (Layer 2) állítja. Megtiltja a fegyverváltást a védekezés ideje alatt.

StopGuarding()

Leírás: Kikapcsolja a védekező állapotot. Visszaállítja a játékos eredeti mozgási sebességét, újra engedélyezi a támadást és a fegyverváltást, valamint visszaállítja az animációs réteget az alapértelmezettre.

```
private void StartGuarding()
{
    speedBeforeGuard = StatsManager.Instance.speed;

    StatsManager.Instance.speed /= 3f;

    combat.enabled = false;
    guard.enabled = true;

    playerAnimator.SetLayerWeight(0, 0f);
    playerAnimator.SetLayerWeight(1, 0f);
    playerAnimator.SetLayerWeight(2, 1f);

    guard.ActivateGuard();
    changeEquipment.canChangeWeapon = false;
}

private void StopGuarding()
{
    StatsManager.Instance.speed = speedBeforeGuard;

    guard.DeactivateGuard();
    guard.enabled = false;
    combat.enabled = true;

    playerAnimator.SetLayerWeight(0, 1f);
    playerAnimator.SetLayerWeight(2, 0f);
    changeEquipment.canChangeWeapon = true;
}
```

41. ábra Player_ChangeToGuard Script

ResetShieldStatus() / OnEnable() / OnDisable()

Leírás: Ezek a metódusok a szkript ki- és bekapcsolásakor, illetve az állapot kényszerített alaphelyzetbe állításakor futnak le. Leállítják a futó folyamatokat, visszaállítják a mozgási sebességet és a pajzs töltöttségi mutatóját, biztosítva, hogy ne ragadjon be a karakter védekező módba.

ResetAfterBlock() / DelayedReset (Coroutine)

Leírás: Akkor hívódik meg, ha a pajzs sikeresen kivédett egy ütést. Egy rövid türelmi idő (shieldGracePeriod) után kényszeríti a védekezés befejezését és elindítja a pajzs újratöltési folyamatát (cooldown).

Visszatérési érték: IEnumerator (a késleltetéshez).

CooldownRoutine (Coroutine)

Leírás: Vezérli a pajzs újratöltődését. Ideiglenesen letiltja a védekezés lehetőségét, majd egy ciklus segítségével fokozatosan feltölti a UI csúszkát a várakozási idő alatt. Amint a folyamat véget ér, a pajzs újra használhatóvá válik.

Visszatérési érték: IEnumerator.

```
public void ResetAfterBlock()
{
    if (isCooldownActive) return;

    StartCoroutine(DelayedReset());
}

IEnumerator DelayedReset()
{
    yield return new WaitForSeconds(shieldGracePeriod);

    if (!isCooldownActive)
    {
        StopAllCoroutines();
        StartCoroutine(CooldownRoutine());
    }
}
```

42. ábra Player_ChangeToGuard Script

2.4.9. Player_Heal

Start() / OnEnable()

Leírás: A szkript indulásakor vagy az objektum ismételt bekapcsolásakor hívódnak meg. Biztosítják, hogy a gyógyítási rendszer tiszta állapottal induljon a ResetHealStatus metódus futtatásával.

ResetHealStatus()

Leírás: Alaphelyzetbe állítja a gyógyítási folyamatot. Leállítja az összes futó Cooldown-t, engedélyezi a gyógyítást, és elrejt a felületről a töltősávot (alpha értéket 0-ra állítja).

Heal()

Leírás: Végrehajtja a játékos gyógyítását, amennyiben a képesség éppen elérhető (canheal). Frissíti a játékos életerejét a StatsManager értékei alapján, láthatóvá teszi a töltősávot, majd elindítja a várakozási időt.

ResetAfterHeal()

Leírás: Elindítja a gyógyítás utáni újratöltődési folyamatot (CooldownRoutine), feltéve, hogy az nem fut már aktívan.

CooldownRoutine (Coroutine)

Leírás: Kezeli a gyógyítási képesség visszatöltődési idejét. Ideiglenesen letiltja a képesség használatát, és egy ciklus segítségével folyamatosan frissíti a healSlider vizuális visszajelzését a képernyőn. A töltési idő (healCooldown) lejártá után elrejt a csúszkát és újra engedélyezi a gyógyítást.

Visszatérési érték: IEnumerator.

2.4.10. Player_Rage

Awake()

Leírás: Az objektum létrejöttkor hívódik meg. Inicializálja a SpriteRenderer referenciát, amely a karakter vizuális visszajelzéséért (színváltozásért) felel a Rage mód alatt.

Start() / OnEnable() / OnDisable()

Leírás: Kezeli a szkript életciklusát. Az indításkor és bekapcsoláskor a ResetRageStatus hívásával alaphelyzetbe állítják a rendszert. Kikapcsoláskor (OnDisable) biztosítják, hogy a karakter színe visszaálljon az eredeti fehérre, elkerülve, hogy a karakter "vörös" maradjon.

Update()

Leírás: Minden képkockánál ellenőrzi a Rage mód aktív időtartamát. Ha a sebezhetetlenség aktív, csökkenti a hátralévő időt és frissíti a hozzá tartozó csúszkát. Amint az idő lejár, automatikusan meghívja a StopInvincibility metódust.

UseSkill()

Leírás: Ez a publikus metódus indítja el a képességet. Ellenőrzi, hogy a játékos nem sebezhetetlen-e már, illetve, hogy a képesség nincs-e éppen újratöltés (cooldown) alatt. Ha mindkét feltétel teljesül, aktiválja a Rage módot.

```
private void Update()
{
    if (durationLeft > 0)
    {
        durationLeft -= Time.deltaTime;
        if (durationSlider != null) durationSlider.value = durationLeft;
    }
    else if (isInvincible)
    {
        StopInvincibility();
    }
}

public void UseSkill()
{
    if (!isInvincible && !isCooldownActive)
    {
        StartInvincibility();
    }
}
```

43. ábra Player_Rage Script

ResetRageStatus()

Leírás: Teljesen alaphelyzetbe állítja a Rage rendszert. Kikapcsolja a sebezhetetlenséget, leállítja a folyamatban lévő visszaszámlálót (Coroutines), elrejt a UI elemeket (Slider), és visszaállítja a karakter színét.

StartInvincibility()

Leírás: Aktiválja a sebezhetetlenséget. Beállítja az időtartamot, láthatóvá teszi a Rage-csúszkát, és a karakter színét vörösre színezi, jelezve a játékosnak a különleges állapotot.

StopInvincibility()

Leírás: Kikapcsolja a sebezhetetlenséget. Visszaállítja a karakter színét és a UI elemeket, majd azonnal elindítja a képesség újratöltődési folyamatát (CooldownRoutine).

CooldownRoutine (Coroutine)

Leírás: Vezérli a képesség használata utáni kényszerpihenőt. Megjeleníti az újratöltési sávot, majd egy ciklusban folyamatosan frissíti annak értékét az invincibilityCooldown időtartama alatt. Amikor a számláló véget ér, elrejt a sávot és újra használhatóvá teszi a képességet.

Visszatérési érték: IEnumerator.

```
private void StartInvincibility()
{
    isInvincible = true;
    durationLeft = invincibilityDuration;

    if (durationGroup != null) durationGroup.alpha = 1;
    if (spriteRenderer != null) spriteRenderer.color = new Color(0.6f, 0f, 0f, 0.9f);
}

private void StopInvincibility()
{
    isInvincible = false;
    durationLeft = 0;
    if (durationGroup != null) durationGroup.alpha = 0;
    if (spriteRenderer != null) spriteRenderer.color = Color.white;

    StartCoroutine(CooldownRoutine());
}
```

44. ábra Player_Rage Script

2.4.11. StatsManager

Awake()

Leírás: Beállítja a Singleton mintát. Biztosítja, hogy az egész játék alatt csak egyetlen StatsManager példány létezzen, így más szkriptek (például a mozgás vagy a harc) bárhol könnyen elérhetik a globális statisztikákat a StatsManager.Instance hivatkozással.

Start()

Leírás: A játék indulásakor alaphelyzetbe állítja a felhasználói felületet és a képességeket. Alapértelmezetten kikapcsolja a pajzsot, az íjászatot, a gyógyítást és a Rage módot, valamint letiltja a hozzájuk tartozó gombokat, amíg azokat a játékos fel nem oldja.

UpdateMaxHealth() / UpdateHealth()

Leírás: Kezeli a játékos életerejének változásait. Az UpdateMaxHealth a maximum értéket növeli, míg az UpdateHealth az aktuális életerőt módosítja (gyógyulás vagy sebzés), ügyelve arra, hogy az ne haladja meg a maximumot. Mindkét metódus azonnal frissíti a képernyőn látható szöveget.

Paraméterek: int amount: A változás mértéke.

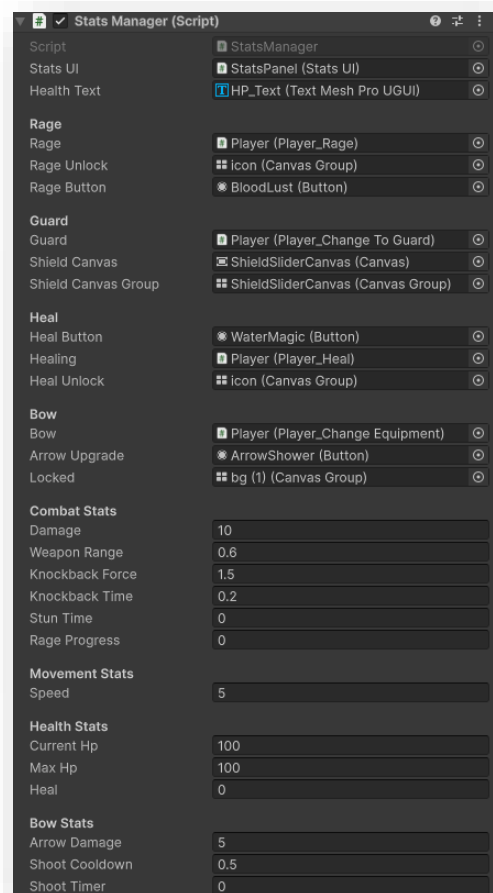
```
public void UpdateMaxHealth(int amount)
{
    maxHp += amount;
    healthText.text = "HP: " + currentHp + "/" + maxHp;
}

public void UpdateHealth(int amount)
{
    currentHp += amount;

    if (currentHp >= maxHp)
    {
        currentHp = maxHp;
    }

    healthText.text = "HP: " + currentHp + "/" + maxHp;
}
```

46. ábra StatsManager Script



45. ábra StatsManager Properties

UpdateAttackDamage() / UpdateSpeed() / UpdateArrow()

Leírás: Statisztikai értékeket módosító segédfüggvények. Lehetővé teszik a kard sebzésének, a karakter mozgási sebességének, vagy a nyílveaszók erejének és újratöltési idejének tartós fejlesztését (például szintlépéskor).

Paraméterek: * int amount / dmg, float cd: A hozzáadandó értékek vagy levonandó időközök.

UnlockGuard() / UnlockArchery() / Heal ()

Leírás: Különböző játékelemeket oldanak fel. Engedélyezik a védekezést, az íjászatot vagy a gyógyítási képességet, aktiválják a hozzájuk tartozó szkripteket, láthatóvá teszik a UI elemeiket, és beállítják a gombok interakcióit.

Paraméterek: Változó (pl. a Heal-nél a gyógyítás mértéke).

UnlockRage()

Leírás: Kezeli a Rage (düh) mód feloldási folyamatát. Minden híváskor növeli a haladási szintet (rageProgress). Amint a játékos eléri az 5. szintet, a képesség véglegesen aktiválódik, a gombja használhatóvá válik, és a "Locked" jelzés eltűnik.

```
public void UnlockArchery()
{
    bow.enabled = true;
    arrowUpgrade.interactable = true;
    locked.alpha = 0;
}

public void UnlockRage()
{
    rageProgress+=1;
    if (rageProgress==5)
    {
        rage.enabled = true;
        rageButton.interactable = true;
        rageUnlock.alpha = 0;
    }

    Debug.Log(rageProgress);
}
```

48. ábra StatsManager Script

```
public void UpdateSpeed(float amount)
{
    speed += amount;
}

public void UnlockGuard()
{
    guard.enabled = true;
    shieldCanvas.enabled = true;
    shieldCanvasGroup.alpha = 1;
}

public void Heal(int amount)
{
    heal += amount;
    healing.enabled = true;
    healButton.interactable = true;
    healUnlock.alpha = 0;
}
```

47. ábra StatsManager Script

2.4.12. StatsUI

Start()

Leírás: Az objektum indulásakor hívódik meg. Azonnal lefuttatja a statisztikák frissítését, hogy a kezelőfelület (UI) már az első megnyitás előtt a helyes adatokat tartalmazza.

OpenStats()

Leírás: Megnyitja a statisztikai ablakot. Megállítja a játékidőt (`Time.timeScale = 0`), növeli az aktív ablakok számát az `UIManager`-ben, és lejátsza a felugró ablak hangeffektusát. Ezzel párhuzamosan láthatóvá teszi a `CanvasGroup`-ot, és engedélyezi az interakciót (kattintásokat) az ablakon.

CloseStats()

Leírás: Bezárja a statisztikai ablakot. Visszaállítja a játékidőt a normál sebességre, lejátsza a záró hangeffektust, és elrejtja a kezelőfelületet. Kikapcsolja az interakciókat is, hogy az ablak ne blokkolja a kattintásokat, amíg rejtve van.

UpdateAllStats()

Leírás: Frissíti a statisztikai ablakban található szöveges mezőket a `StatsManager`-ben tárolt aktuális értékek alapján. A listában szereplő slotok sorrendben a sebzést (`Strength`), a mozgási sebességet (`Agility`) és a maximális életerő tizedét (`Vitality`) jelenítik meg.

```
public void OpenStats()
{
    if (statsOpen) return;
    statsOpen = true;
    UIManager.OpenWindowCount++;
    Time.timeScale = 0;

    AudioManager.instance.Play("PopUpOpen");

    UpdateAllStats();
    statsCanvas.alpha = 1;
    statsCanvas.interactable = true;
    statsCanvas.blocksRaycasts = true;
}

public void CloseStats()
{
    if (!statsOpen) return;
    statsOpen = false;
    UIManager.OpenWindowCount--;
    Time.timeScale = 1;

    AudioManager.instance.Play("PopUpClose");

    statsCanvas.alpha = 0;
    statsCanvas.interactable = false;
    statsCanvas.blocksRaycasts = false;
}
```

49. ábra StatsUI Script

```
public void UpdateAllStats()
{
    statsSlots[0].GetComponentInChildren<TMP_Text>().text = "Strength: " + StatsManager.Instance.damage;
    statsSlots[1].GetComponentInChildren<TMP_Text>().text = "Agility: " + StatsManager.Instance.speed.ToString("0.##");
    statsSlots[2].GetComponentInChildren<TMP_Text>().text = "Vitality: " + StatsManager.Instance.maxHp / 10;
}
```

50. ábra StatsUI Script

2.4.13. Enemy_Movement

Start()

Leírás: Az ellenség létrejöttkor fut le. Inicializálja a belső referenciákat (Rigidbody2D, Animator), megkeresi a "Base" taggel ellátott bázis objektumot, és beállítja a karakter kezdeti állapotát Idle (tétlen/bázis felé tartó) módba.

Update()

Leírás: Minden képkockánál ellenőrzi az ellenség állapotát. Ha nem áll fenn korlátozó tényező (hátralökés vagy bázis elleni támadás), folyamatosan futtatja a játékos keresését (CheckForPlayer), kezeli a támadási töltési időt, és a jelenlegi állapot alapján mozgatja a karaktert a játékos vagy a bázis irányába.

ApplySeparationForce (FixedUpdate)

Leírás: A fizikai frissítéskor (FixedUpdate) hívódik meg. Egy "lágú ütközésvizsgálatot" végez a többi ellenséggel: ha túl közel vannak egymáshoz, egy tolóerőt alkalmaz rájuk, hogy elkerüljék a sprite-ok teljes egymásba csúszását (csoportos mozgás finomítása).

Move()

Leírás: Kiszámítja és alkalmazza a mozgást a megadott célpont (target) felé. Kezeli a karakter megfordulását (Flip) a vízszintes irány alapján, és a célirányba mutató sebességet ad a Rigidbody-nak.

Flip()

Leírás: Megfordítja az ellenség grafikáját a vízszintes tengelyen a localScale módosításával, és frissíti a facingDirection változót.

Paraméterek: Transform target: Az objektum, amely felé az ellenségnek haladnia kell.

```
void Move(Transform target)
{
    if (target == null) return;

    if (target.position.x > transform.position.x && facingDirection == -1 || target.position.x < transform.position.x && facingDirection == 1)
    {
        Flip();
    }

    Vector2 direction = (target.position - transform.position).normalized;
    rb.linearVelocity = direction * speed;
}

void Flip()
{
    facingDirection *= -1;
    transform.localScale = new Vector3(transform.localScale.x * -1, transform.localScale.y, transform.localScale.z);
}
```

51. ábra Enemy_Movement Script

CheckBaseArrival() / AttackBaseCoroutine()

Leírás: Figyeli, hogy az ellenség elérte-e a bázist. Ha a távolság elég kicsi, elindítja a támadási folyamatot: az ellenség megáll, egy rövid várakozás után sebzést okoz a bázisnak, részecskeeffektet vált ki, majd megsemmisíti önmagát.

Visszatérési érték: IEnumerator (a késleltetett támadáshoz).

CheckForPlayer()

Leírás: Egy kör alakú detektálási zónát használ a játékos keresésére. Ha a játékos a hatótávon belülre kerül, az ellenség üldözőbe veszi (Chasing). Ha elég közel ér hozzá és a támadási időzítő lejárt, támadó állapotba vált. Ha a játékos elhagyja a zónát, az ellenség visszatér a bázis felé tartó állapotba.

ChangeState()

Leírás: Az ellenség állapotgépének (FSM) központi kezelője. Átváltáskor alaphelyzetbe állítja az összes animációs paramétert, beállítja az új állapotot, és aktiválja a megfelelő animációs trigger (Idle, Chasing, Attacking, Dash).

Paraméterek: EnemyState newState: Az új állapot, amibe az ellenségnek lépnie kell.

ResetToIdle() / ResetToChasing()

Leírás: Segédfüggvények (gyakran animációs események hívják meg), amik visszaállítják az ellenséget alaphelyzetbe vagy üldöző módba egy akció (például támadás) befejezése után.

```
public void ChangeState(EnemyState newState)
{
    if (enemyState == newState) return;

    anim.SetBool("IsIdle", false);
    anim.SetBool("IsChasing", false);
    anim.SetBool("IsAttacking", false);
    anim.SetBool("IsDashing", false);

    enemyState = newState;

    if (enemyState == EnemyState.Idle) anim.SetBool("IsIdle", true);
    else if (enemyState == EnemyState.Chasing) anim.SetBool("IsChasing", true);
    else if (enemyState == EnemyState.Attacking) anim.SetBool("IsAttacking", true);
    else if (enemyState == EnemyState.Dash) anim.SetBool("IsDashing", true);
}
```

52. ábra Enemy_Movement Script

2.4.14. Enemy_Health

Start()

Leírás: Az ellenség létrejöttkor hívódik meg. Beállítja az aktuális életerőt a maximumra, és frissíti az életerőcsíkot (UpdateHealthBar), hogy a vizuális visszajelzés szinkronban legyen az adatokkal a játék kezdetén.

ChangeHealth()

Leírás: Módosítja az ellenség életerejét. Ha az életerő 0 alá esik, lekezeli a halált: elsüti az OnMonsterDefeated eseményt a tapasztalati pont (EXP) kiosztásához, hozzáadja az aranyat a játékos inventory-jához, részecskeeffektet (deathParticle) hoz létre, majd megsemmisíti az ellenség objektumát.

Paraméterek: int amount: Az életerő változásának mértéke (pozitív érték gyógyít, negatív érték sebez).

UpdateHealthBar()

Leírás: Frissíti az ellenség feje felett látható életerőcsík (Slider) értékét. A csúszka pozícióját az aktuális és a maximális életerő hányadosa alapján számítja ki.

LateUpdate()

Leírás: Minden képkocka végén lefut. Ez a metódus felel azért, hogy az életerőcsík vizuálisan ne forduljon meg (ne tükröződjön), amikor maga az ellenség megfordul. Korrigálja a Slider localScale értékét az ellenség irányának megfelelően.

2.4.15. Enemy_Combat

Attack()

Leírás: Végrehajtja az ellenség támadását (általában egy animációs esemény hívja meg). Egy kör alakú ütközésvizsgálatot (OverlapCircleAll) végez a megadott fegyvertávolságon belül a játékos rétegén. Ha találatot észlel, és a játékos aktív, alkalmazza a hátralökési erőt (Knockback) a mozgás szkripten keresztül, valamint levonja a sebzést a játékos életerejéből.

OnDrawGizmosSelected()

Leírás: Segédfüggvény a Unity szerkesztő (Editor) számára. Kirajzol egy piros drótvázaskört az ellenség köré, amely a fegyver hatótávolságát szemlélteti. Ez a vizuális segítség csak akkor látható a Scene nézetben, ha az ellenség objektuma ki van jelölve, segítve a fejlesztőt a hatótáv pontos beállításában.

```
public void Attack()
{
    Collider2D[] hits = Physics2D.OverlapCircleAll(transform.position, weaponRange, playerLayer);
    if (hits.Length > 0)
    {
        GameObject player = hits[0].gameObject;

        if (player.activeInHierarchy)
        {
            Player_Movement movement = player.GetComponent<Player_Movement>();
            if (movement != null)
            {
                movement.Knockback(transform, knockbackForce, stunTime);
            }

            Player_Health health = player.GetComponent<Player_Health>();
            if (health != null)
            {
                health.ChangeHealth(-damage);
            }
        }
    }
}
```

53. ábra Enemy_Combat Script

2.4.16. Enemy_Knockback

Start()

Leírás: Az objektum indulásakor fut le. Inicializálja a belső referenciákat: eltárolja a Rigidbody2D komponenst a fizikai mozgathoz, valamint az Enemy_Movement szkriptet az ellenség állapotának kezeléséhez.

Knockback()

Leírás: Aktiválja az ellenség hátralökődését (például ha a játékos megüti). Átállítja az ellenség állapotát EnemyState.Knockback-re, kiszámítja a lökés irányát a támadó pozíciójából, és fizikai erőt fejt ki a Rigidbody-ra. Ezzel párhuzamosan elindítja a bénulási folyamatot kezelő Coroutine-t.

Paraméterek:

- **Transform forceTransform:** A lökés forrásának (például a játékosnak) a pozíciója.
- **float knockbackForce:** A hátralökés ereje.
- **float knockbackTime:** Az időtartam, amíg a lökés ereje hat a karakterre.
- **float stunTime:** A hátralökés utáni bénulás hossza, amíg az ellenség még nem tud mozogni.

StunTimer (Coroutine)

Leírás: Két szakaszban kezeli az ellenség mozgásképtelenségét. Először megvárja a hátralökődés végét, majd nullázza az ellenség sebességét. Ezután kivárja a bénulási időt (stun), végül visszaváltja az ellenséget Idle (tétlen) állapotba, amivel újra engedélyezi a normál mozgást és üldözést.

Paraméterek:

- **float knockbackTime:** A mozgási energia megszűnéséig tartó várakozás.
- **float stunTime:** A teljes bénulás feloldásáig tartó várakozás.

Visszatérési érték: IEnumerator.

2.4.17. SkillTreeToggler

Start()

Leírás: Az objektum indulásakor fut le. Összekapcsolja a gombokat a megfelelő funkciókkal: az enter gombhoz hozzárendeli a képességfa megnyitását, a leave gombhoz pedig annak bezárását (Event Listener-ek beállítása).

SkillTreeEnter()

Leírás: Megnyitja a képességfa felületét. Aktiválja a skillsCanvas-t (láthatóvá teszi és engedélyezi a kattintásokat), ezzel egy időben pedig elrejti és kikapcsolja a statisztikákat jelző statsCanvas-t. Növeli az aktív ablakok számát és lejátssza a nyitási hangeffektust.

SkillTreeLeave()

Leírás: Bezárja a képességfa felületét és visszavált a statisztikák nézetre. Kikapcsolja a képességfa láthatóságát és interakcióit, visszahozza a statisztikai ablakot, csökkenti az aktív ablakok számlálóját, valamint lejátssza a zárási hangeffektust.

```
private void Start()
{
    if (enter != null) enter.onClick.AddListener(SkillTreeEnter);
    if (leave != null) leave.onClick.AddListener(SkillTreeLeave);
}

public void SkillTreeEnter()
{
    if (skillTreeOpen) return;
    skillTreeOpen = true;
    UIManager.OpenWindowCount++;

    AudioManager.instance.Play("PopUpOpen");

    skillsCanvas.alpha = 1;
    skillsCanvas.blocksRaycasts = true;
    skillsCanvas.interactable = true;

    statsCanvas.alpha = 0;
    statsCanvas.blocksRaycasts = false;
    statsCanvas.interactable = false;
}

public void SkillTreeLeave()
{
    if (!skillTreeOpen) return;
    skillTreeOpen = false;
    UIManager.OpenWindowCount--;

    AudioManager.instance.Play("PopUpClose");

    skillsCanvas.alpha = 0;
    skillsCanvas.blocksRaycasts = false;
    skillsCanvas.interactable = false;

    statsCanvas.alpha = 1;
    statsCanvas.blocksRaycasts = true;
    statsCanvas.interactable = true;
}
```

54. ábra SkillTreeToggler Script

2.4.18. SkillSlot

OnValidate()

Leírás: Ez a metódus a Unity szerkesztőn (Editor) belül fut le, amikor az objektum értékei megváltoznak az Inspector ablakban. Biztosítja, hogy a képesség ikonja és a szintjelző szöveg azonnal frissüljön a szerkesztőben is, amennyiben a szükséges referenciák hozzá vannak rendelve.

TryUpgradeSkill()

Leírás: Megkísérli a képesség szintjének növelését. Ellenőrzi, hogy az aktuális szint alatta van-e a ScriptableObject-ben meghatározott maximális szintnek. Siker esetén növeli a szintet, elsüti az OnAbilityPointSpent eseményt (amely értesíti a rendszert az elköltött pontról), majd frissíti a kezelőfelületet.

UpdateUI()

Leírás: Frissíti a képesség vizuális megjelenítését. Beállítja a gombot interaktívra, lecseréli a képesség ikonját a skillSO-ban tárolt sprite-ra, és kiírja az aktuális szintet a maximumhoz viszonyítva (például: "1/5").

```
private void OnValidate()
{
    if (skillSO != null && skillLevelText != null && skillButton != null)
    {
        UpdateUI();
    }
}

public void TryUpgradeSkill()
{
    if (currentLevel < skillSO.maxLevel)
    {
        currentLevel++;
        OnAbilityPointSpent?.Invoke(this);
        UpdateUI();
    }
}

private void UpdateUI()
{
    skillButton.interactable = true;
    skillIcon.sprite = skillSO.skillIcon;
    skillLevelText.text = currentLevel.ToString() + "/" + skillSO.maxLevel.ToString();
}
```

55. ábra SkillSlot Script

2.4.19. SkillTreeManager

OnEnable() / OnDisable()

Leírás: Kezeli az eseményfeliratkozásokat. Bekapcsoláskor feliratkozik a képességpontok elköltésére (OnAbilityPointSpent) és a szintlépésre (OnLevelUp). Kikapcsoláskor leiratkozik ezekről, hogy megelőzze a memóriaszivárgást és a felesleges hívásokat, amikor a képességfa nem aktív.

Start()

Leírás: A játék indításakor inicializálja a rendszert. Végigmegy az összes képességhelyen (SkillSlot), és dinamikusan hozzárendel egy eseményfigyelőt a gombjaikhoz, amely kattintáskor ellenőrzi az elérhető pontokat. Végül frissíti a pontokat kijelző szöveget.

CheckAvailablePoints()

Leírás: Ellenőrzi, hogy a játékosnak van-e elkölthető képességpontja. Ha a pontok száma nagyobb, mint nulla, engedélyezi a kiválasztott képesség fejlesztését a TryUpgradeSkill metódus meghívásával.

Paraméterek: SkillSlot slot: Az a konkrét képességhely, amit a játékos fejleszteni kíván.

HandleAbilityPointsSpent()

Leírás: Akkor hívódik meg, amikor egy képesség sikeresen szintet lépett. Levon egy pontot a játékos keretéből az UpdateAbilityPoints metóduson keresztül, biztosítva, hogy a pontok száma és a kijelzés szinkronban maradjon.

Paraméterek: SkillSlot skillSlot: A fejlesztett képesség (az eseményből érkezik).

UpdateAbilityPoints()

Leírás: Módosítja az aktuálisan elérhető képességpontok számát a megadott mennyiséggel, majd frissíti a felhasználói felületen (pointsText) látható feliratot. Ez a metódus fut le szintlépéskor (pont hozzáadása) és fejlesztéskor (pont levonása) is.

Paraméterek: int amount: A hozzáadandó vagy levonandó pontok mennyisége.

2.4.20. SkillManager

OnEnable / OnDisable

Leírás: Kezeli az eseményekre való fel- és leiratkozást. Amikor a szkript aktívvá válik, elkezd figyelni a SkillSlot.OnAbilityPointSpent eseményt. Ez biztosítja, hogy a játék azonnal reagáljon, amint a játékos elkölt egy képességpontot. A leiratkozás (OnDisable) segít megelőzni a memóriahibákat az objektum megsemmisülésekor.

HandleAbilityPointSpent

Leírás: Ez a központi feldolgozó metódus, amely meghatározza, hogy mi történjen egy képesség fejlesztésekor. Egy switch szerkezet segítségével azonosítja a képesség nevét a ScriptableObject-ből (skillSO.skillName), és végrehajtja a hozzá tartozó statisztikai módosítást a StatsManager-en keresztül.

Paraméterek: SkillSlot slot: Az a konkrét képességhely, amelynek a fejlesztése kiváltotta az eseményt.

A switch szerkezet ágai (Példák)

Leírás: A metóduson belül minden képességtípus saját logikát kapott:

- **Max Health Boost:** Megnöveli a játékos maximális életerejét 10 egységgel.
- **Attack Damage Boost:** Fix 5 ponttal növeli a kard sebzését.
- **Speed Boost:** Kismértékben (0.2) gyorsítja a karakter mozgását.
- **Arrow Buff:** Növeli a nyílvezzők sebzését és csökkenti a lövések közötti időt.
- **Unlock típusok (Guard, Archery, Rage):** Feloldják az adott harci modult vagy képességet a játékban.
- **Water Magic:** Aktiválja/fejleszti a gyógyítási statisztikát.

```
private void HandleAbilityPointSpent(SkillSlot slot)
{
    string skillName = slot.skillSO.skillName;

    switch (skillName)
    {
        case "Max Health Boost":
            StatsManager.Instance.UpdateMaxHealth(10);
            break;
        case "Attack Damage Boost":
            StatsManager.Instance.UpdateAttackDamage(5);
            break;
        case "Speed Boost":
            StatsManager.Instance.UpdateSpeed(0.2f);
            break;
        case "Arrow Buff":
            StatsManager.Instance.UpdateArrow(5, 0.1f);
            break;
        case "Guard Unlock":
            StatsManager.Instance.UnlockGuard();
            break;
        case "Archery Unlock":
            StatsManager.Instance.UnlockArchery();
            break;
        case "Water Magic":
            StatsManager.Instance.Heal(5);
            break;
        case "Bloodlust Magic":
            StatsManager.Instance.UnlockRage();
            break;
        default:
            Debug.Log("Unknown skill: " + skillName);
            break;
    }
}
```

56. ábra SkillManager Script

2.5. Algoritmusok leírása (Pseudokód)

Az alábbiakban a két legösszetettebb logikai folyamat leírása látható algoritmus-leíró eszközökkel.

2.5.1 Szintlépési és Tapasztalati Pont Rendszer (ExpManager)

Ez az algoritmus kezeli a pontok gyűjtését és a szintlépési küszöb dinamikus növekedését.

Pseudo-kód:

ALGORITMUS GainExperience(mennyiség)

aktuálisExp = aktuálisExp + mennyiség

CIKLUS AMÍG aktuálisExp >= expToLevel

LevelUp()

VÉGE CIKLUS

UI_Frissítése()

VÉGE ALGORITMUS

ALGORITMUS LevelUp()

szint = szint + 1

aktuálisExp = aktuálisExp - expToLevel

expToLevel = kerekít(expToLevel * expGrowth)

Esemény_Kiváltása(OnLevelUp)

VÉGE ALGORITMUS

2.5.2 Fegyverváltás és Animációs Rétegek (Player_ChangeEquipment)

Ez a rész az állapotok közötti váltást és a vizuális rétegek (Layers) kezelését mutatja be.

Pszeudo-kód:

ALGORITMUS SwitchWeaponMode()

HA *új_aktív* AKKOR

Közelharc_Szkript = BE

Íj_Szkript = KI

Animáció_Réteg(0) *Súly* = 1.0 // Alap réteg

Animáció_Réteg(1) *Súly* = 0.0 // Íjász réteg

EGYÉBKÉNT

Közelharc_Szkript = KI

Íj_Szkript = BE

Animáció_Réteg(0) *Súly* = 0.0

Animáció_Réteg(1) *Súly* = 1.0

Lövés_Időzítő = *Újratöltési_Idő*

VÉGE HA

Váltási_Hang_Lejátszása()

VÉGE ALGORITMUS

2.5.3 Ellenség Mesterséges Intelligencia (Enemy_Movement)

Az ellenség döntési mechanizmusa: játékos üldözése vagy a bázis támadása.

Pszeudo-kód:

ALGORITMUS UpdateAI()

HA állapot == Knockback VAGY állapot == AttackingBase AKKOR

MEGSZAKÍT // Nem mozog

VÉGE HA

Játékos_Keresése_Körben(detektálásiSugár)

HA Játékos_Található AKKOR

Távolság = Távolság(Én, Játékos)

HA Távolság <= TámadásiTávolság ÉS Időzítő <= 0 AKKOR

Állapot = Támadás

Támadás_Végrehajtása()

EGYÉBKÉNT HA Távolság > TámadásiTávolság AKKOR

Állapot = Üldözés

Mozgás(Játékos_Felé)

VÉGE HA

EGYÉBKÉNT

Állapot = Idle (Bázis felé tartás)

Mozgás(Bázis_Felé)

HA Távolság(Én, Bázis) <= ElérésiTávolság AKKOR

Bázis_Támadása_És_Önmegsemmisítés()

VÉGE HA

VÉGE HA

VÉGE ALGORITMUS

2.5.4. Képességheloldás Logika (SkillManager / SkillTreeManager)

A képességheloldás elköltésének és a hatások érvényesítésének folyamata.

Pszeudo-kód:

ALGORITMUS TryUpgradeSkill(kiválasztottSlot)

HA elérhetőPontok > 0 AKKOR

HA kiválasztottSlot.szint < kiválasztottSlot.maxSzint AKKOR

kiválasztottSlot.szint = kiválasztottSlot.szint + 1

elérhetőPontok = elérhetőPontok - 1

ELÁGAZÁS (kiválasztottSlot.neve) ALAPJÁN:

"Max Health Boost": Életerő_Növelése(10)

"Guard Unlock": Védekezés_Feloldása()

"Archery Unlock": Íjászat_Feloldása()

// ... többi eset

VÉGE ELÁGAZÁS

UI_Frissítése()

VÉGE HA

VÉGE HA

VÉGE ALGORITMUS

2.6. Tesztesetek bemutatása

2.6.1. Normál teszteset: Szintlépés és Képességfeloldás

- **Cél:** Ellenőrizni, hogy a tapasztalati pontok gyűjtése kiváltja-e a szintlépést és kapunk-e elkölthető pontot.
- **Felhasználói tevékenység:** Az "Enter" billentyű megnyomása (teszt jelleggel 2000 EXP-t ad).
- **Program reakciója:** * Az ExpManager növeli a szintet.
 - A SkillTreeManager növeli az availablePoints értékét.
 - A UI-on a "Points: 1" felirat jelenik meg.
- **Teendő:** A játékos megnyithatja a képességfát és elköltheti a pontot.

2.6.2. Extrém teszteset: "Bolondbiztosság" – Képesség túlhúzása

- **Cél:** Megakadályozható-e, hogy a felhasználó a maximális szint fölé fejlesszen egy képességet, ha sok pontja van.
- **Felhasználói tevékenység:** Egy 5/5 szintű képességre (pl. Max Health Boost) való ismételt kattintás, miközben még van szabad képességpontja.
- **Program reakciója:** * A SkillSlot.TryUpgradeSkill() metódus ellenőrzi a currentLevel < skillSO.maxLevel feltételt.
 - Mivel a feltétel hamis, a kód nem fut tovább.
 - A képességpontok száma nem csökken, a szint nem nő 6-ra.
- **Üzenet/Visszajelzés:** Nincs változás a UI-on, a gomb kattintható marad, de nem történik esemény.
- **Teendő:** Nincs teendő, a rendszer sikeresen megvédte az adatstruktúrát a hibás beviteltől.

2.6.3 Határérték tesztet: Pajzs és Életerő interakció

- **Cél:** Mi történik, ha a játékos életeréje 1 egység, és a pajzsát használja egy erős támadás ellen?
- **Felhasználói tevékenység:** A "Guard" gomb (Shift) nyomva tartása az ellenség támadása pillanatában.
- **Program reakciója:** * A Player_ChangeToGuard aktiválódik.
 - A sebzés nem az életerőt (HP) csökkenti, hanem aktiválja a ResetAfterBlock() metódust.
 - A pajzs Cooldown állapotba kerül (Slider lemegy 0-ra).
- **Üzenet/Visszajelzés:** A pajzs UI csúszkája kiürül és elkezd töltődni.
- **Teendő:** A játékosnak el kell futnia vagy fegyvert kell váltania, amíg a pajzs újra nem töltődik.

2.6.4 A tesztelés során kiderült hibák felsorolása

A fejlesztés során az alábbi kritikus hibák (bugok) kerültek rögzítésre és javításra:

Hiba leírása	Oka	Megoldás
Negatív életerő: Az ellenség HP-ja mínuszba ment halál után.	Hiányzott a 0-ra való kerekítés vagy az objektum azonnali törlése.	ChangeHealth metódusban if (currentHp <= 0) feltétel hozzáadása és Destroy(gameObject).
"Megfordult" UI: Ha az ellenség balra nézett, az életerőcsíkja is tükröződött (fordítva látszott a szöveg).	A Slider a karakter gyermeke volt, így átvette a negatív X skálázást.	Az Enemy_Health szkriptbe bekerült a LateUpdate korrekció, ami fixálja a Slider irányát.
Pontszám eltolódás: Szintlépéskor több pontot vont le a rendszer, mint amennyit adott.	Az eseménykezelő kétszer futott le az OnEnable miatt.	Az OnDisable metódusban megfelelően leiratkoztunk az eseményekről.

2.6.5. Alkalmazott tesztelési módszertanok

Fekete doboz (Black-box) tesztelés

Ezt főként a játékegyensúly (balansz) tesztelésére használtuk. A tesztelő nem látta a kódot, csak játszott a programmal.

- *Példa:* Az új sebzése túl gyenge volt a közelharc kardhoz képest, ezért a SkillManager-ben növeltük az "Arrow Buff" értékét.

Fehér doboz (White-box) tesztelés

A kód belső szerkezetének vizsgálata. Itt ellenőriztük a Singleton minták (StatsManager.Instance) helyes működését.

- *Példa:* Annak tesztelése, hogy ha két StatsManager kerül a színtérre (Scene), az Awake metódus valóban megsemmisíti-e a másodikat a Destroy(gameObject) hívással.

Regressziós tesztelés

Minden új képesség (pl. Rage mód) hozzáadása után újra végigfuttattuk az alapvető mozgás- és harci teszteket, hogy megbizonyosodjunk róla: az új kód nem rontotta el a meglévő funkciókat (pl. a düh mód alatt is működik-e a gyógyulás).

2.7. Jövőbeli fejlesztési lehetőségek

A projekt skálázhatósága érdekében az alábbi irányokba érdemes továbbvinni a fejlesztést:

2.7.1. Mesterséges Intelligencia és Ellenség-típusok

A jelenlegi "követlek és ütlek" logika mellett szükség lenne változatosabb viselkedési formákra:

- **Távolsági ellenségek:** Olyan íjász vagy mágus ellenfelek, amelyek tartják a távolságot a játékostól.
- **Boss-mechanikák:** Egyedi fázisokkal és speciális képességekkel rendelkező főellenségek, amelyeknél nem elég a nyers erő, hanem taktikára (pl. pajzshasználat időzítése) van szükség.

2.2. Kibővített Képességfa (Skill Tree)

A jelenlegi lineáris fejlesztések helyett elágazó ágakat lehetne létrehozni:

- **Specializációk:** A játékos dönthetne, hogy a "Tank" (életerő, pajzs), az "Assassin" (sebesség, kritikus sebzés) vagy a "Ranger" (íjászat, távolsági extrák) irányba fejleszti a karakterét.
- **Aktív képességek:** Passzív statisztika-növelők helyett új gombnyomásra aktiválható skilllek (pl. területre ható csapás, rövid távú teleport/dash).

2.3. Környezeti interakciók és Világépítés

- **Interaktív objektumok:** Robbanó hordók, csapdák vagy lassító mocsaras területek a pályán, amelyeket a játékos és az ellenség is kihasználhat.
- **Napszakok változása:** Egy egyszerű időhurok, ahol éjszaka az ellenségek erősebbek vagy agresszívbak, nappal pedig a gyógyulás gyorsabb.

Összegzés

A záródolgozat elkészítése egy összetett folyamat volt, amely nemcsak technikai tudásomat mélyítette el, hanem rávilágított a rendszerszemléletű gondolkodás fontosságára is a szoftverfejlesztésben. Az alábbiakban összefoglalom a projekt során szerzett tapasztalataimat és a jövőre vonatkozó céljaimat.

1. Szakmai és személyes fejlődés

A fejlesztés során a legjelentősebb előrelépést az **(OOP)** gyakorlati alkalmazásában értem el. A Unity motor és a C# nyelv használata során megtanultam számos számomra új dolgot.

Személyes területen a projekt megtanított a **fegyelmezett időbeosztásra** és a dokumentáció fontosságára. Rájöttem, hogy egy jól megírt metódusleírás vagy folyamatábra sokszor értékesebb a fejlesztés későbbi szakaszában, mint maga a nyers forráskód.

2. Kihívások a fejlesztés során

A projekt nem volt mentes a nehézségektől, amelyek közül a legnagyobbak a következők voltak:

- **Állapotgépek kezelése:** Az ellenségek mesterséges intelligenciájának (Enemy_Movement) összehangolása – hogy mikor kell üldözni, mikor támadni, és mikor reagálni a hátralökésre – komoly logikai tervezést igényelt.

- **Balanszolás:** Megtalálni az egyensúlyt a játékos fejlődése és az ellenségek erősödése között, hogy a játékmenet se túl könnyű, se frusztrálóan nehéz ne legyen.

3. Jövőbeli célok

A záródolgozat befejezése után világossá vált számomra, hogy a szoftverfejlesztés, azon belül is a játékfejlesztés az a terület, amellyel professzionális szinten szeretnék foglalkozni. Jövőbeli céljaim:

Összességében a projekt megerősített abban, hogy a szoftverfejlesztés nem csupán kódolás, hanem folyamatos tanulás és alkotás. A záródolgozat során szerzett tudás stabil alapot biztosít a szakmai karrierem megkezdéséhez, a felmerült nehézségek pedig értékes tapasztalatként szolgálnak a jövőbeni komplex feladatok megoldásához.

Forrásmegjelölés

1. Online szakmai dokumentációk

Unity Technologies: *Unity Manual & Scripting API.*

oForrás: <https://docs.unity3d.com/Manual/index.html>

oLeírás: A Rigidbody2D, Collision2D és az Animator rendszerek implementálásához használt hivatalos referencia.

Microsoft Learn: *C# Guide - Documentation.*

oForrás: <https://learn.microsoft.com/en-us/dotnet/csharp/>

oLeírás: A delegáltak (delegate), események (event) és a generikus listák használatának elméleti háttere.

2. Elektronikus források és oktatóanyagok

Brackeys (YouTube csatorna): *How to make a 2D Melee Attack in Unity.*

- Elérhetőség: <https://www.youtube.com/@Brackeys>
- Leírás: Az OverlapCircleAll módszerrel történő találatérzékelés logikai felépítése.

NightRun Studio (YouTube csatorna): *Unity 2D Tutorials - RPG Mechanics.*

oElérhetőség: <https://www.youtube.com/@NightRunStudio>

oLeírás: A csatorna videóit alapvető segítséget nyújtottak a **Képességfa (Skill Tree)** vizuális felépítésében, a **ScriptableObject** alapú adatkezelésben, valamint az ellenségek **AI állapotgépének (State Machine)** logikai strukturálásában.

3. Felhasznált szoftverek és eszközök

Unity Hub / Unity Editor (6000.3.9f1 LTS): A játék motorja és fejlesztői környezete.

Visual Studio 2022: Integrált fejlesztői környezet a C# szkriptek írásához.

Unity Asset Store: A sprite-ok és részecskeeffektek (deathParticle) forráshelye.

4. A projekt GitHub-adattárának elérhetősége:

<https://github.com/matepiee/one-knight-army>

Ábrajegyzék

1. ábra Letöltés	8
2. ábra Kicsomagolás	8
3. ábra Elindítás.....	8
4. ábra Főmenü	9
5. ábra Start Game gomb.....	10
6. ábra Settings gomb	10
7. ábra Exit gomb	11
8. ábra Lebegő játékcím.....	11
9. ábra Beállítások menü.....	12
10. ábra Exit gomb	12
11. ábra Grafics Quality Selector.....	13
12. ábra Music Volume Slider	14
13. ábra SFX Volume Slider	14
14. ábra A Hős	15
15. ábra Kard Csapás.....	16
16. ábra Íjász Mód.....	16
17. ábra Íj Lövés	16
18. ábra Guard/Block.....	17
19. ábra Blokk Rúd	17
20. ábra Pálya	18
21. ábra Életerő jelző.....	18
22. ábra Bázis Életerő Jelző	19
23. ábra Tapasztalat Szint Jelző.....	19
24. ábra Ezköztár	19
25. ábra Arany Jelző	19
27. ábra Tornyok	20
26. ábra Várfal.....	20
28. ábra Csata Indítása Gomb.....	20
29. ábra Boltos	21
30. ábra Ezköztár	22
31. ábra Bolt Megnyitva.....	22
32. ábra Statisztika	23

33. ábra Talentumfa	25
34. ábra Gyógyító Képesség Gomb	26
35. ábra Düh Képesség Jelző	26
36. ábra Düh Képesség Gomb	26
37. ábra Player Movement Script	33
38. ábra Player Attack Script	34
39. ábra ExpManager Script	36
40. ábra Player_ChangeEquipment Script	37
41. ábra Player_ChangeToGuard Script	40
42. ábra Player_ChangeToGuard Script	41
43. ábra Player_Rage Script	43
44. ábra Player_Rage Script	44
45. ábra StatsManager Properties	45
46. ábra StatsManager Script	45
47. ábra StatsManager Script	46
48. ábra StatsManager Script	46
49. ábra StatsUI Script	47
50. ábra StatsUI Script	47
51. ábra Enemy_Movement Script	48
52. ábra Enemy_Movement Script	49
53. ábra Enemy_Combat Script	51
54. ábra SkillTreeToggler Script	53
55. ábra SkillSlot Script	54
56. ábra SkillManager Script	56